

Talbaser og data serialisering

Netværk og distribueret design

Oversigt

- Uge 1
 - Mandag: Talbaser og data serialisering
 - Tirsdag: OSI og TCP
 - Onsdag: UDP og realtidsspil
 - Torsdag: Realtids opgave forsat (selvstudie)
 - Ingen undervisning – ingen mødepligt
 - Fredag: Distribueret design
- Uge 2
 - Mandag: Sikkerhed og Kryptering
 - Tirsdag: REST
 - Onsdag: Virtualisering

Eksamen

- Ingen dedikeret netværks eksamen
- Forløbet udgør en del af programmerings eksamen
 - Som er mundtlig
 - Hvor i trækker et spørgsmål
 - ...og der er altid en netværks del i spørgsmålet.
 - I bør have øvelser/projekter fra undervisningen i kan trække på

Dagsorden

- Talbaser og logiske binære operationer
 - Øvelser
- Serialisering
 - Øvelse – tekstbaseret serialisering
- Binær serialisering
 - Øvelse binær serialisering

Talbaser og Aritmetik - decimal

- Vores "normale" talbase bliver også kaldt for decimal eller base10
- Vi har adgang til 10 tal (0-9), og når vi kommer over 9 går vi til 0 igen, og tilføjer et 1 til venstre
- Kan nedbrydes i hvor mange gange man har 1, 10 100 osv.
- F.eks $235 = 2 * 10^2(100'ere) + 3 * 10^1(10'ere) + 5 * 10^0(1'ere)$
- I c# kendes det simpelt som;

```
int number = 10;
```

Binær



binary

/ˈbʌɪn(ə)rɪ/

adjective

1. relating to, composed of, or involving two things.
2. relating to, using, or denoting a system of numerical notation that has 2 rather than 10 as a base.

- Også kendt som base2
- Hvor vi normalt (i base10) kan tælle til 10 kan vi kun tælle til 2 (0 og 1)
- Vi har adgang til 2 tal (0-1), og når vi kommer over 1 går vi til 0 igen, og tilføjer et 1 til venstre
 - Prøv evt. med windows' lommeregner
- Nedbrudt siger vi ikke hvor mange 1,10 og 100... har vi, men hvor mange 1,2,4... har vi
 - Med andre ord: hvor mange $2^0(1)$, $2^1(2)$, $2^2(4)$... har vi.
- 235 bliver derfor til **11101011** i base2
- $1 * 128(2^7) + 1 * 64(2^6) + 1 * 32(2^5) + 0 * 16(2^4) + 1 * 8(2^3) + 0 * 4(2^2) + 1 * 2(2^1) + 1 * 1(2^0) = 235$

Binær

- Normal denoteres talbasen som subscript efter tallet så som: **11101011₂** - betyder det det skal læses i binær
- alt inde bag ved scenerne på computeren foregår i binær (tændt eller slukket)
- I c# kan vi også skrive vores variable ved at bruge "0b" foran tallet i binær:
 - `int binaryNumber = 0b11101011;`
 - Præcis det samme som at skrive:
 - `int binary = 235;`

Binær – variable størrelse

- Når vi snakker om data på computeren snakker vi jo altid bits og bytes
- Det er her vigtigt at huske at en byte består af 8 bits (0 eller 1) og har derfor en maximal værdi på
- $11111111_2 = 2^8_{10} - 1 = 255_{10}$
 - Dvs 256 forskellige værdier når vi inkluderer 0, og det gør vi selvfølgelig ☺
- Der er derfor ikke noget der stopper os fra at snakke om 4bit værdier, så ville vi have en udtrykskræft på $16_{10} = (2^4)$ med største værdi 15_{10}
 - $= 1111_2 = (1 * 2^3 + 1 * 2^2 + 1 * 2^1 + 1 * 2^0)_{10}$
- På samme kender vi også de maximale størrelser på andre datatyper, kun ud fra hvor mange bit de er.
- Eks. er en unsigned int 32bit (eller 4 bytes), og dens maksimale værdi er derfor:
- 4294967295, altså en udtrykskræft på 2^{32} med en maximal værdi på $2^{32} - 1$ (da vi er 0 indekseret)

C# og bytes

Logic operators

logik og operatorer – komplement ~

- ~ komplement operator, vender alle bits om, altså 0 bliver til 1 og omvendt.

```
byte b1 = 0b00101011;
byte res = (byte)~b1;
string op = "~";
string b1AsString = Convert.ToString(b1, toBase: 2).PadLeft(8, '0');
string resAsString = Convert.ToString(res, toBase: 2).PadLeft(8, '0');
Console.WriteLine("-----");
Console.WriteLine("Binary   | Operator | Decimal");
Console.WriteLine("-----");
Console.WriteLine(String.Format("{0,-8} | {1,-8} | {2,5}", $"{b1AsString}", $"{op.PadLeft(5)}", $"{b1}"));
Console.WriteLine(String.Format("{0,-8} | {1,-8} | {2,5}", $"{resAsString}", "", $"{res}"));
Console.WriteLine("-----");
```

Microsoft Visual Studio Debug Console

Binary	Operator	Decimal
00101011	~	43
11010100		212

logik og operatorer – left shift <<

- << er left shift, og vil sige at vi rykker alle bits n gange til venstre
- I nedenstående tilfælde rykker vi 1 gang til venstre
 - Samme som at gange med 2!
 - Egentlig er det $2^1 = 2$ det bliver ganget med
 - Hvor 1 er antallet af shifts!
 - $43 << 3 = 43 * 2^3 = 43 * 8$

```
byte b1 = 0b00101011;
int n = 1;
byte res = (byte)(b1 << n);
string op = $"<<{n}";
string b1AsString = Convert.ToString(b1, toBase: 2).PadLeft(8, '0');
string resAsString = Convert.ToString(res, toBase: 2).PadLeft(8, '0');
Console.WriteLine("-----");
Console.WriteLine("Binary | Operator | Decimal");
Console.WriteLine("-----");
Console.WriteLine(String.Format("{0,-8} | {1,-8} | {2,5}", $"{b1AsString}", $"{op.PadLeft(5)}", $"{b1}"));
Console.WriteLine(String.Format("{0,-8} | {1,-8} | {2,5}", $"{resAsString}", "", $"{res}"));
Console.WriteLine("-----");
```

Microsoft Visual Studio Debug Console

```
-----
Binary | Operator | Decimal
-----
00101011 | <<1 | 43
01010110 | | 86
-----
```

logik og operatorer – right shift - >>

- >> er right shift, og vil sige at vi rykker alle bits n gange til højre
- I nedenstående eksempel rykker vi 2 gange til højre
 - Samme som at dividere med 2^2 (rundet ned)

Når du deler med 2 x kan man lave et kommatal, vil man afrunde til nærmeste

```
byte b1 = 0b00101011;
int n = 2;
byte res = (byte)(b1>>n);
string op = ">>{n}";
string b1AsString = Convert.ToString(b1, toBase: 2).PadLeft(8, '0');
string resAsString = Convert.ToString(res, toBase: 2).PadLeft(8, '0');
Console.WriteLine("-----");
Console.WriteLine("Binary   | Operator | Decimal");
Console.WriteLine("-----");
Console.WriteLine(String.Format("{0,-8} | {1,-8} | {2,5}", $"{b1AsString}", $"{op.PadLeft(5)}", $"{b1}"));
Console.WriteLine(String.Format("{0,-8} | {1,-8} | {2,5}", $"{resAsString}", "", $"{res}"));
Console.WriteLine("-----");
```

Microsoft Visual Studio Debug Console

Binary	Operator	Decimal
00101011	>>2	43
00001010		10

logik og operatorer – logical AND - &

- Ligesom vores “&&” til normale if statements ved bools.
- Ved tal bibeholdes alle de bits hvor begge er 1, 0 ellers

AND		RES	
b1	b2		
0	0	0	
0	1	0	
1	0	0	
1	1	1	

Sandhedstabel for “&”

```
byte b1 = 0b00101011;
byte b2 = 0b10101001;
byte res = (byte)(b1&b2);
string op = "&";
string b1AsString = Convert.ToString(b1, toBase: 2).PadLeft(8, '0');
string b2AsString = Convert.ToString(b2, toBase: 2).PadLeft(8, '0');
string resAsString = Convert.ToString(res, toBase: 2).PadLeft(8, '0');
Console.WriteLine("-----");
Console.WriteLine("Binary | Operator | Decimal");
Console.WriteLine("-----");
Console.WriteLine(String.Format("{0,-8} | {1,-8} | {2,5}", $"{b1AsString}", $"{op.PadLeft(5)}", $"{b1}"));
Console.WriteLine(String.Format("{0,-8} | {1,-8} | {2,5}", $"{b2AsString}", $"{op.PadLeft(5)}", $"{b2}"));
Console.WriteLine(String.Format("{0,-8} | {1,-8} | {2,5}", $"{resAsString}", $"{op.PadLeft(5)}", $"{res}"));
Console.WriteLine("-----");
```

Microsoft Visual Studio Debug Console

```
-----
Binary | Operator | Decimal
-----
00101011 | & | 43
10101001 | & | 169
00101001 | & | 41
-----
```

logik og operatorer – logical OR - |

- Bibeholder alle de bits hvor mindst den ene bit er 1, 0 ellers
- Præcis ligesom vores "||" til normale if statements.

OR		RES
b1	b2	
0	0	0
0	1	1
1	0	1
1	1	1

Sandhedstabel for "||"

```
byte b1 = 0b00101011;
byte b2 = 0b10101001;
byte res = (byte)(b1|b2);
string op = "|";
string b1AsString = Convert.ToString(b1, toBase: 2).PadLeft(8, '0');
string b2AsString = Convert.ToString(b2, toBase: 2).PadLeft(8, '0');
string resAsString = Convert.ToString(res, toBase: 2).PadLeft(8, '0');
Console.WriteLine("-----");
Console.WriteLine("Binary   | Operator | Decimal");
Console.WriteLine("-----");
Console.WriteLine(String.Format("{0,-8} | {1,-8} | {2,5}", $"{b1AsString}", $"{op.PadLeft(5)}", $"{b1}"));
Console.WriteLine(String.Format("{0,-8} | {1,-8} | {2,5}", $"{b2AsString}", $"{op.PadLeft(5)}", $"{b2}"));
Console.WriteLine(String.Format("{0,-8} | {1,-8} | {2,5}", $"{resAsString}", $"{op.PadLeft(5)}", $"{res}"));
Console.WriteLine("-----");
```

Microsoft Visual Studio Debug Console

```
-----
Binary   | Operator | Decimal
-----
00101011 |         |      43
10101001 |         |     169
10101011 |         |     171
-----
```

logik og operatorer – logical XOR – “^”

- Exclusive or
- 1 på de bits hvor de er forskellige, 0 hvis ens
- Kan ikke sammenlignes med normale if statement

XOR		RES
b1	b2	
0	0	0
0	1	1
1	0	1
1	1	0

Sandhedstabel for “^”

```
byte b1 = 0b00101011;
byte b2 = 0b10101001;
byte res = (byte)(b1^b2);
string op = "^";
string b1AsString = Convert.ToString(b1, toBase: 2).PadLeft(8, '0');
string b2AsString = Convert.ToString(b2, toBase: 2).PadLeft(8, '0');
string resAsString = Convert.ToString(res, toBase: 2).PadLeft(8, '0');
Console.WriteLine("-----");
Console.WriteLine("Binary | Operator | Decimal");
Console.WriteLine("-----");
Console.WriteLine(String.Format("{0,-8} | {1,-8} | {2,5}", $"{b1AsString}", $"{op.PadLeft(5)}", $"{b1}"));
Console.WriteLine(String.Format("{0,-8} | {1,-8} | {2,5}", $"{b2AsString}", $"{op.PadLeft(5)}", $"{b2}"));
Console.WriteLine(String.Format("{0,-8} | {1,-8} | {2,5}", $"{resAsString}", $"{op.PadLeft(5)}", $"{res}"));
Console.WriteLine("-----");
```

Microsoft Visual Studio Debug Console

```
-----
Binary | Operator | Decimal
-----
00101011 | ^ | 43
10101001 | ^ | 169
10000010 | ^ | 130
-----
```

Operator rækkefølge

- Rækkefølgende disse bliver udført i er som følger:
 1. Bitwise complement operator $\sim$
 2. Shift operators $\ll$ and $\gg$
 3. Logical AND operator $\&$
 4. Logical exclusive OR operator $\wedge$
 5. Logical OR operator $|$
- Man kan bruge parenteser, hvis man ønsker noget skal udføres i anderledes rækkefølge.
- Alle disse operationer kan også udføres på ints og bytes i decimal, computeren er ligeglad.

Bit flags

- Vi kan bruge binære data som mere end bare én værdi
 - Vi kan repræsentere hver bit som en "flag" for om noget er gældende
 - 4 bits til at fortælle hvilke af WASD er blevet trykket
 - 0110 = A og S er trykket på
- Eksempel på brugen af binære værdi som bit flags:
 - <https://pressbooks.lib.jmu.edu/programmingpatterns/chapter/bitflags/>

Binær – negative tal

- Alt hvad vi har gennemgået nu er kendt som unsigned værdier, hvilket betyder vi kun kan repræsentere positive tal
- Hvis vi gerne vil have negative tal, bruger vi den venstremest bit til at fortælle os om det er et positivt eller negativt tal. Denne værdi er kendt som "most significant bit"(msb)

Binær – negative tal

- For at lave negative numre også kendt som **signed** værdier, skal vi gennem en håndfuld steps
 1. Find det positive nummer der skal repræsenteres - f.eks -12, $12 = 0000\ 1100$
 2. Find komplementet: $\sim 0000\ 1100 = 1111\ 0011$
 3. Adder 1 til dette nummer: $1111\ 0011 + 1 = 1111\ 0100$
 4. Omregn og lad msb være negativ
 - $-128 + 64 + 32 + 16 + 4 = -12$
- Da vi skal bruge 1 bit til at bestemme positiv eller negativ har en signed byte en udtrykskræft på $[-127, 127]$, hvilket længden på også er præcis 8bit = 255
- I c# bruger vi dog "–" operatoren som normalt til negative binære numre
 - `int negativeNumber = -0b00001100;`

Flere talbaser - HexaDecimal

- Også kendt som base 16 normalt detoneret ved et “#” eller “0x” prefix i C#
- Helt ligesom base2 og base10 læser vi fra højre mod venstre
- Men vi har kun 10 tal (0-9), så hvordan repræsenterer vi base16?
- Efter 9, bruger vi bogstaver (a-f), for at repræsentere 10-15
 - Vores tal går derfor fra 0-F:
 - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f

Hexadecimal - eksempel

- Lad os tage et konkret eksempel
 - $\#FF1493 = 15 * 16^5 + 15 * 16^4 + 1 * 16^3 + 4 * 16^2 + 9 * 16^1 + 3 * 16^0 = 16716947$
- Vi kunne også skrive den som 3 separate tal på en byte hver:
 - $\#FF, \#14, \#93$ hvilket giver os 255,20,147
- Værdier i dette format ringer måske en klokke, da det oftest er en af disse formater man bruger til at repræsentere farver i RGB på vores farve, $FF1493$ er:
 - Hvor R = FF, G = 14, B = 93
- Det smarte ved Hexadecimal er vi kan repræsentere store tal på få karakterer
- I c# kan vi skrive hex ved prefixet "0x"



```
int hex = 0xFF1493;
Console.WriteLine(hex);
```

Microsoft Visual Studio Debug Console

16716947

Endnu flere talbaser

- Der er intet der stopper os fra at have andre talbaser, hvis vi virkelig gerne vil.
- Det er de samme regler der gælder lige gyldigt hvilken talbase vi bruger
- De mest normale er dog:
 - Binær – base2
 - Decimal – base10
 - Octal – base8
 - Hexadecimal – base16

Encoding – fra bytes til bogstaver

- Encoding er princippet at transformere/oversætte sine bytes til karakterer eller den anden vej omkring.
- En simpel form for encoding er ASCII (1 byte i størrelse)

```
byte[] bytes = new byte[] {64,98};  
  
Console.WriteLine($"bytes to encode to ascii {string.Join(",", bytes)}");  
var enCoded = Encoding.ASCII.GetString(bytes);  
Console.WriteLine($"encoded bytes : {enCoded}");  
var deCoded = Encoding.ASCII.GetBytes(enCoded);  
Console.WriteLine($"decoded bytes from ascii: {string.Join(",", deCoded)}");
```

Microsoft Visual Studio Debug Console

```
bytes to encode to ascii 64,98  
encoded bytes : @b  
decoded bytes from ascii: 64,98
```

Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII
64	01000000	100	40	@	96	01100000	140	60	`
65	01000001	101	41	A	97	01100001	141	61	a
66	01000010	102	42	B	98	01100010	142	62	b
67	01000011	103	43	C	99	01100011	143	63	c
68	01000100	104	44	D	100	01100100	144	64	d
69	01000101	105	45	E	101	01100101	145	65	e
70	01000110	106	46	F	102	01100110	146	66	f
71	01000111	107	47	G	103	01100111	147	67	g
72	01001000	110	48	H	104	01101000	150	68	h
73	01001001	111	49	I	105	01101001	151	69	i
74	01001010	112	4A	J	106	01101010	152	6A	j
75	01001011	113	4B	K	107	01101011	153	6B	k
76	01001100	114	4C	L	108	01101100	154	6C	l
77	01001101	115	4D	M	109	01101101	155	6D	m

Udrag fra ASCII table

Encoding – fra bytes til bogstaver

- Der er mange andre former for encoding, hvor tabellerne varierer i størrelse, og dermed antal tegn man kan skrive, på bekostning af at man læser flere bytes adgangen.
- Disse inkluderer:
 - Utf8, som har de første 128 fra ascii, men andre tegn bagefter (bl.a æ, ø og å, som ikke er i ASCII!)
 - Diverse encoding standarder til f.eks japanske og kinesiske tegn
- Så længe man bruger den samme til encode og decode, behøver man ikke tænke så meget over det 😊

Øvelse 1

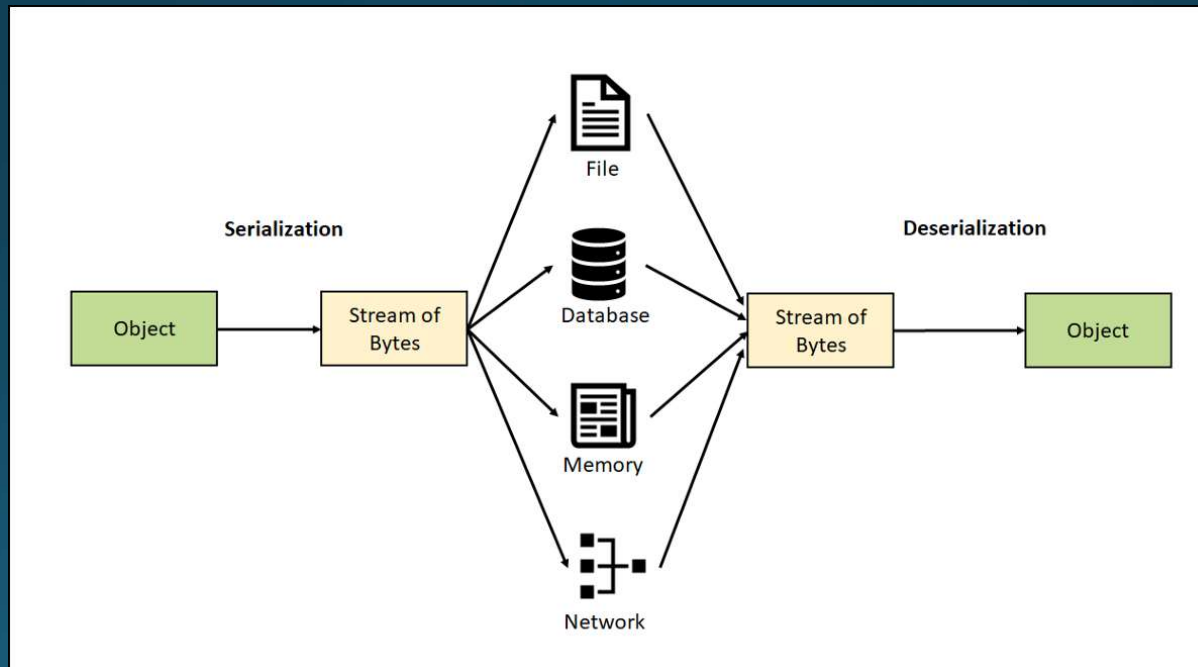
- Skriv de første 10 karakterer af dit navn i ASCII følgende formater:
 - Hexadecimal
 - Binær
- F.eks. JAN:
 - ASCII værdier: 74 65 78
 - Hex værdier: 4A 41 4E
 - Binære værdier: 1001010 01000001 1001110

Øvelse 2

- Udregn følgende under antagelse af unsigned positive tal :
 - $0100\ 0000_2 + 0000\ 1110_2$
 - $1000\ 0000_2 - 4D_{16}$
 - $0111\ 1101_2 \& 1111\ 0100_2$
 - $0010\ 1011_2 \ll 1_{10}$
 - $0110\ 0000_2 | 0000\ 0001_2$
 - $1001\ 0100_2 \wedge 1111\ 0001_2$
 - $148_{16} \gg 2_{10}$
 - $\sim 180_{10}$ (antag 8bit ☺)
- Oversæt til resultaterne til ASCII og få én streng ud
- I bestemmer selv hvordan I udregner dem; online tools, windows calculator, I hånden, I gennem kode.

Data serialisering

- Data serialization er processen med at konvertere dataobjekter til et format, der kan gemmes eller transporteres og senere gendannes tilbage til objekter.



Data serialisering - formål

- Tillader persistens:
 - Data kan gemmes på disken eller i en database og bevares mellem applikationskørsler.
 - Save data til spil etc.
 - Gemme data til fil eller i en database
- Muliggør dataudveksling:
 - Data kan sendes over netværket eller mellem forskellige applikationer/platforme.
- Støtter kommunikation mellem forskellige sprog:
 - Serialization muliggør udveksling af data mellem applikationer skrevet i forskellige programmeringssprog.
 - Ved at bruge en standard ved vi hvordan forskellig data skal læses / skrives.
 - Serialization giver kompatibilitet

Data serialisering - teknikker

- Tekstbaseret serialization: Data repræsenteres som tekst i et menneske læseligt format som f.eks. JSON eller XML.
- Binær serialization: Data konverteres til en binær repræsentation, der er mere kompakt og hurtigere at behandle end tekstbaserede formater.
 - Ikke læseligt af mennesker.

```
http://localhost:8080/Json/SyncReply/Contacts
{
  "Contacts": [
    {
      "FirstName": "Demis",
      "LastName": "Bellot",
      "Email": "demis.bellot@gmail.com"
    },
    {
      "FirstName": "Steve",
      "LastName": "Jobs",
      "Email": "steve@apple.com"
    },
    {
      "FirstName": "Steve",
      "LastName": "Ballmer",
      "Email": "steve@microsoft.com"
    },
    {
      "FirstName": "Eric",
      "LastName": "Schmidt",
      "Email": "eric@google.com"
    },
    {
      "FirstName": "Larry",
      "LastName": "Ellison",
      "Email": "larry@oracle.com"
    }
  ]
}

http://localhost:8080/XML/SyncReply/Contacts
<ContactsResponse xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Contacts>
    <Contact>
      <Email>demis.bellot@gmail.com</Email>
      <FirstName>Demis</FirstName>
      <LastName>Bellot</LastName>
    </Contact>
    <Contact>
      <Email>steve@apple.com</Email>
      <FirstName>Steve</FirstName>
      <LastName>Jobs</LastName>
    </Contact>
    <Contact>
      <Email>steve@microsoft.com</Email>
      <FirstName>Steve</FirstName>
      <LastName>Ballmer</LastName>
    </Contact>
    <Contact>
      <Email>eric@google.com</Email>
      <FirstName>Eric</FirstName>
      <LastName>Schmidt</LastName>
    </Contact>
    <Contact>
      <Email>larry@oracle.com</Email>
      <FirstName>Larry</FirstName>
      <LastName>Ellison</LastName>
    </Contact>
  </Contacts>
</ContactsResponse>
```

Json og XML

```
10001101110001011000110111001011011100111001
00110001001110110001110100111011100010110101
00110111000101100011011100101101110011100100
11000100111011000111010011101110001011010110
00110111000101100011011100101101110011100100
11000100111011000111010011101110001011010110
00011011100010110001101110010110111001110010
01100010011101100011101001110111000101101011
10001101110001011000110111001011011100111001
00110001001110110001110100111011100010110101
11011100010110001101110010110111001110010001
00010011101100011101001110111000101101011010
00110111000101100011011100101101110011100100
11000100111011000111010011101110001011010110
00110111000101100011011100101101110011100100
```

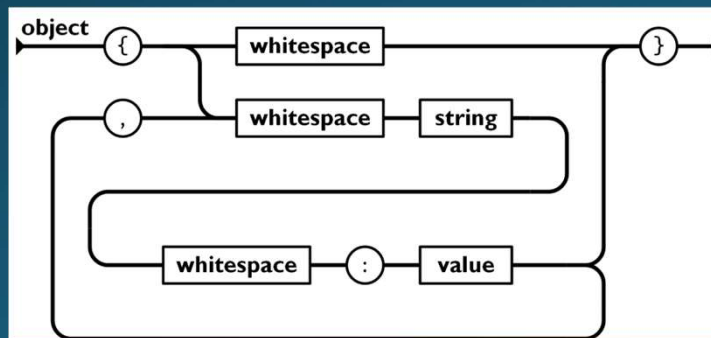
Binær!

Data serialisering - Tekstbaseret

- Tekstbaseret serialisering involverer konvertering af data til en tekstrepræsentation, der kan gemmes eller overføres.
- De mest almindelige tekstbaserede serialization formater inkluderer JSON (JavaScript Object Notation) og XML (eXtensible Markup Language).
- Tekstbaserede formater bruger en syntaks til at repræsentere datastrukturer som nøgle-værdi-par eller hierarkiske strukturer.
 - JSON er nøgle-værdi-par
 - XML er hierarkisk
- Tekstbaseret serialization er let at implementere og forstå, da det bruger almindelige tegn og strukturer, der ligner naturligt sprog.

Json

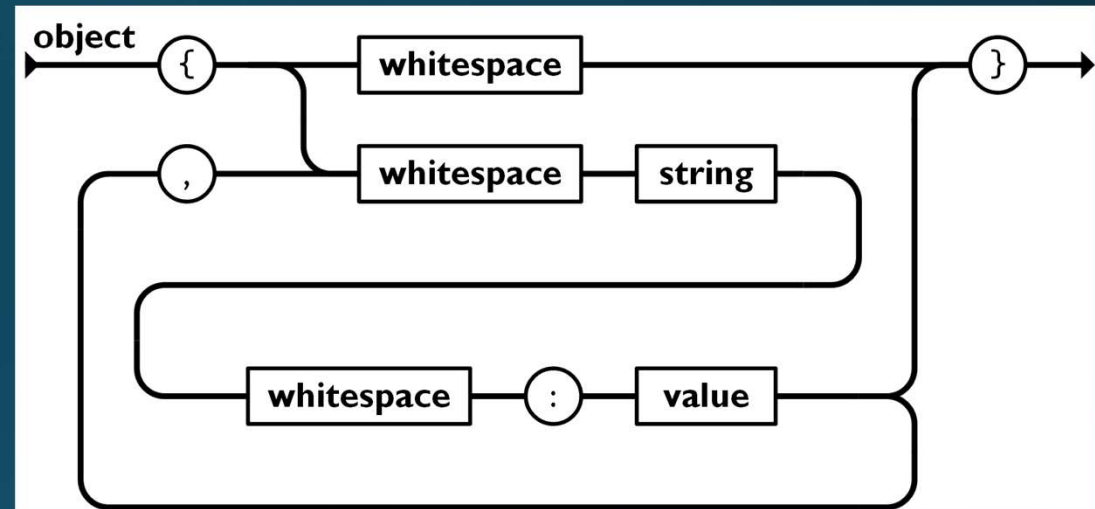
- JavaScript Object Notation
- letvægts dataudvekslingsformat.
- Bruges til repræsentation af strukturerede data som key value pairs.
- Popularitet inden for dataudveksling mellem systemer og sprog.
- Simpelt og menneskelæseligt
- Det største format brugt til web APler.



```
1 {  
2   "LastName": "Doe",  
3   "FirstName": "Jane",  
4   "AccountID": "1469278",  
5   "Mobile": "+49 999999 9999",  
6   "Email": "jane.doe99@example.com",  
7   "ExtensionField1_KUT": "value1",  
8   "ExtensionField2_KUT": "value2"  
9 }
```

Json - object

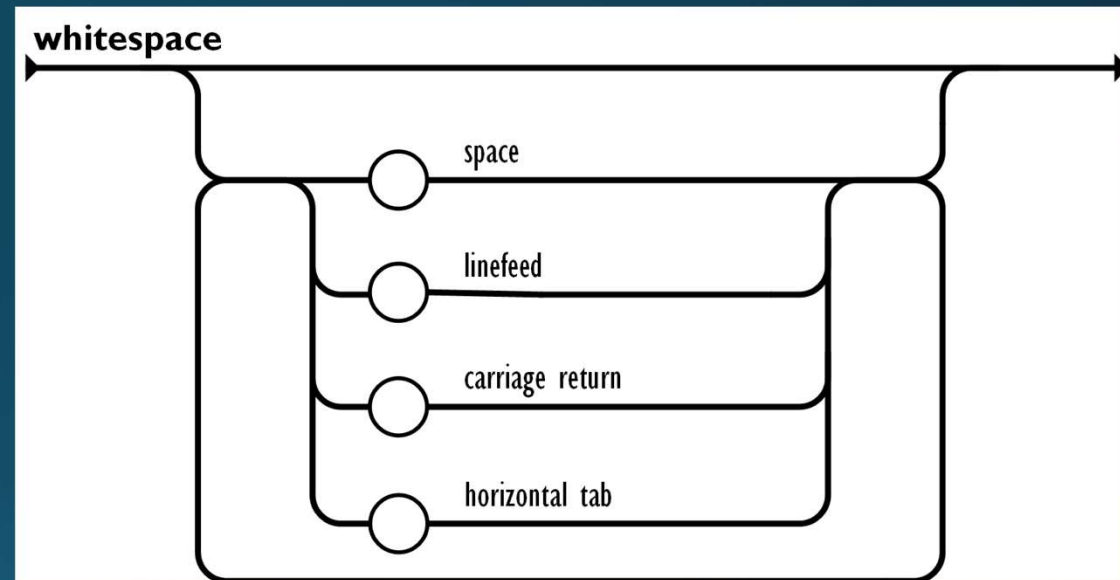
- Det yderste element er altid af Typen "object"
- Startes med {
- Dette efterfølges af en string som key og dernæst en value
- Kan have flere KVP'er adskilt af ,
- Afsluttes med }



```
{  
  "key1": "value1",  
  "key2": "value2",  
  "key3": 123,  
  "key4": true  
}
```

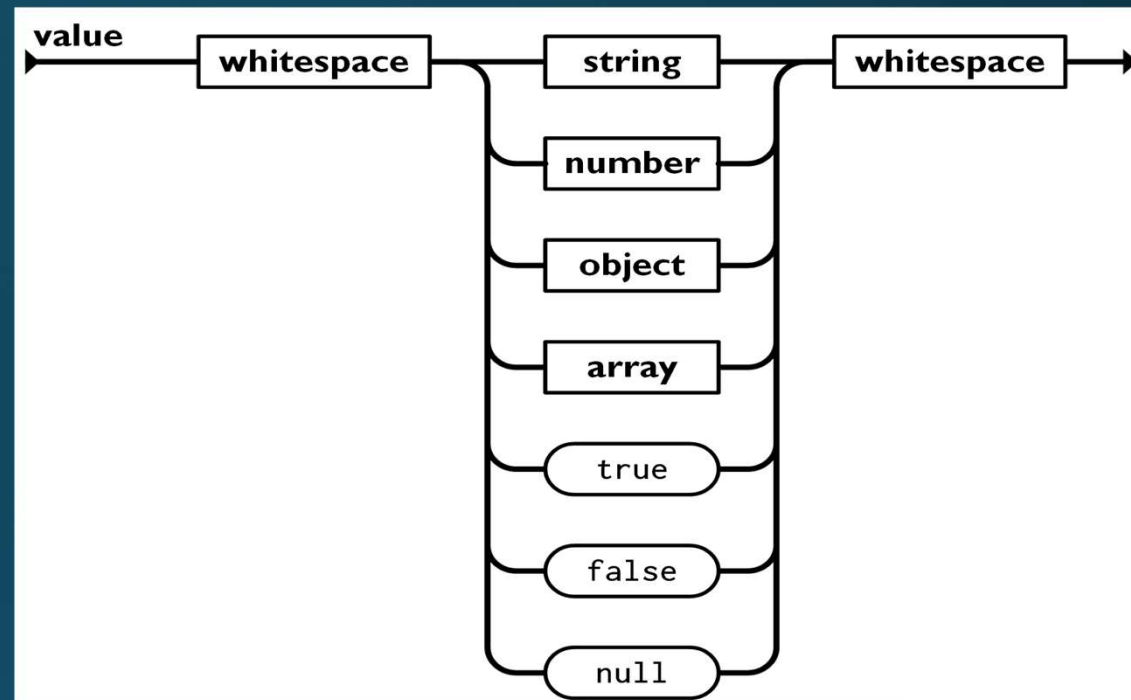
Json - Whitespace

- Space
 - Normale mellemrums tast
- Linefeed
 - Angiver starten på en ny linje
- Carriage return
 - Kontroltegn der flytter markøren til starten af en linje
- Horizontal tab
 - Normale tab tast
- VIGTIGT: disse er kun for at gøre JSON nemmere at læse, og har ingen betydning for datastrukturen.



Json - Value

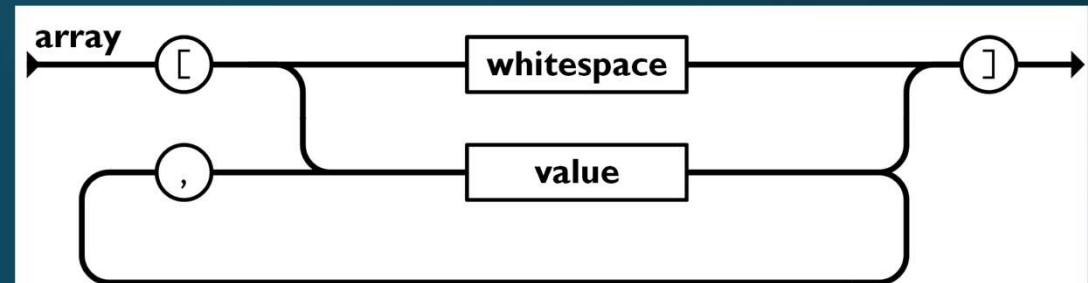
- Alle values vil altid være en af disse typer.
- Bid mærke I at specifikationen understøtter nested objecter da en value kan være af typen object!



```
{  
  "key1": "value1",  
  "key2": "value2",  
  "integer": 123,  
  "float": 123.4,  
  "key4": true,  
  "nestedObject": {  
    "nestedKey1": "nestedValue1",  
    "nestedKey2": "nestedValue2"  
  }  
}
```

Json - Array

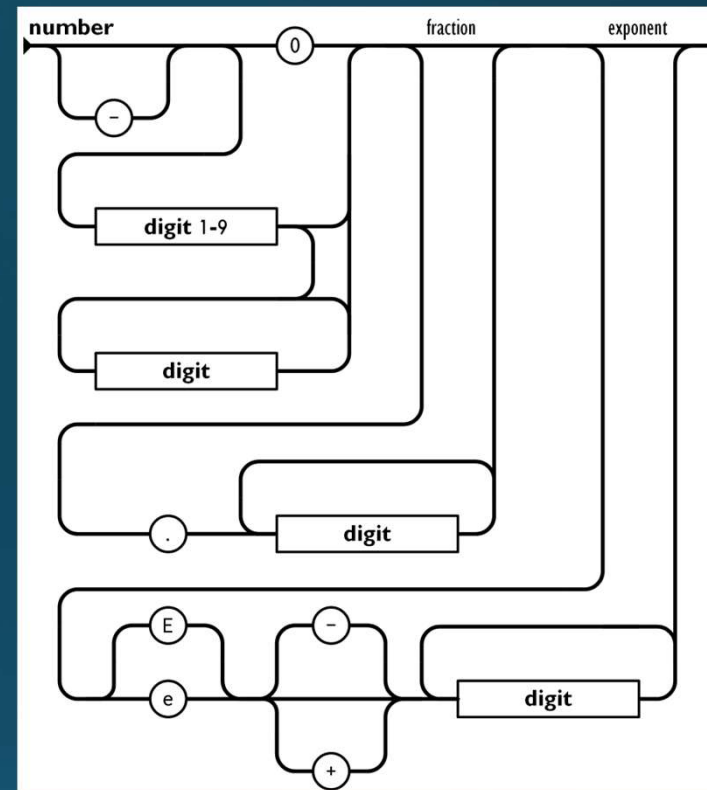
- Startes med [
- Values efterfulgt af ","
- Da det er values kan det selvfølgelig være alt hvad en value kan være.
- Sluttes med]



```
{  
  "name": "John Doe",  
  "age": 30,  
  "isStudent": true,  
  "favoriteColors": ["blue", "green", "red"],  
  "grades": [85, 92, 78, 95],  
  "coordinates": [  
    {"x": 10, "y": 20},  
    {"x": 30, "y": 40},  
    {"x": 50, "y": 60}  
  ]  
}
```

Json - Number

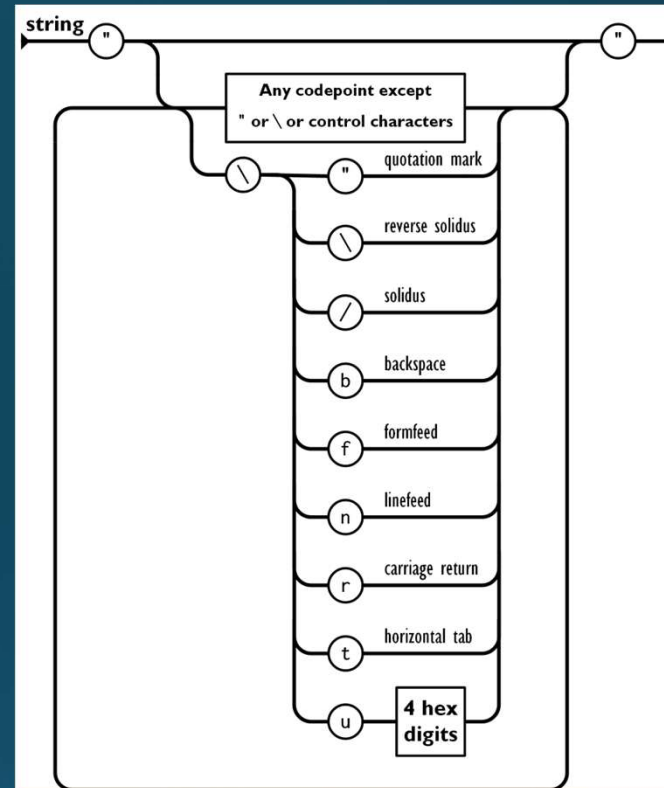
- Understøttelse for både positive og negative heltal og decimal tal
- Operatorene "E" og "e" bliver brugt til scientific notation, og er en måde at repræsentere lange tal på mindre plads.
 - Der er ingen forskel på "e" eller "E"
- Scientific1 bliver til:
 - $1.23E + 10 = 1.23 * 10^{10} = 12300000000$
- Scientific2 bliver til:
 - $-4.56e - 5 = -4.56 * 10^{-5} = -0.0000456$



```
{  
  "integer": 42,  
  "float": 3.14159,  
  "scientific1": 1.23E+10,  
  "scientific2": -4.56e-5,  
  "negativeNumber": -10,  
  "decimal": 123.45,  
}
```


Json - String

- Den ser intimiderende ud, men er blot "normale" characters og escape characters
- Derudover er der escape karakteren \u for at hente værdier udenfor ascii i Unicode.
 - Kunne være \u263A, som er ☺



```
{  
  "QuotesInString": "hello \"world\"",  
  "NewLineInString": "Firstline \nSecondLine",  
  "TabInString": "Firstline \ttabbedLine"  
}
```

Json i C#

- C# har en indbygget JsonSerializer i namespace System.Text.Json
- Man kan serialisere et object ved at bruge metoden:

```
JsonSerializer.Serialize(jsonObject, typeof(JsonObject));
```

▲ 1 of 12 ▼ **string** JsonSerializer.Serialize(**object?** value, **Type** inputType, [JsonSerializerOptions? options = null])
Converts the value of a specified type into a JSON string.
value: The value to convert.

- Tager et simpelt object og laver det til en string i JSON format
- For at få tilbage til sit object fra string kan man bruge:

```
JsonSerializer.Deserialize<>
```

▲ 7 of 16 ▼ **TValue?** JsonSerializer.Deserialize<**TValue**>(string json, JsonSerializerOptions? options = null)
Parses the text representing a single JSON value into an instance of the type specified by a generic type parameter.
TValue: The target type of the JSON value.

- Bruger blot sin json string, og specificerer hvilken type den skal deserialiseres som.

Json i C# Eksempel

```
using System.Text.Json;

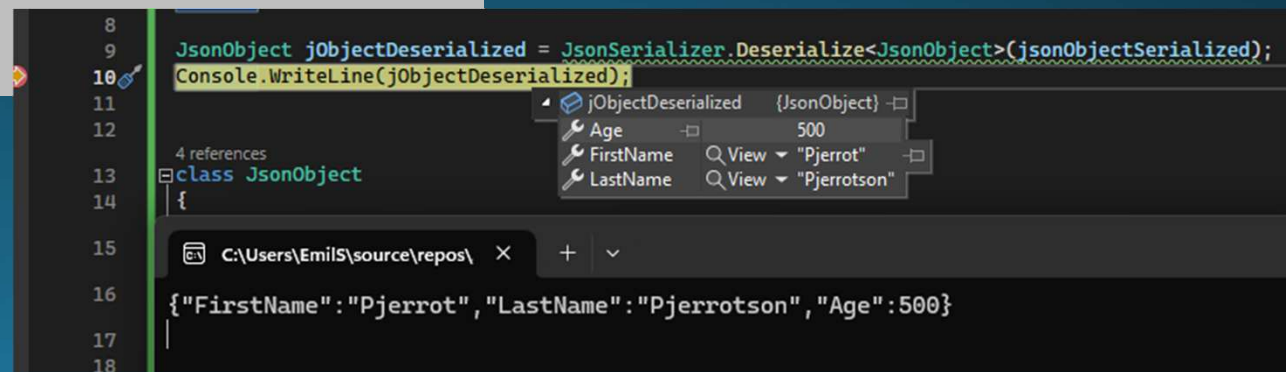
JsonObject jsonObject = new JsonObject()
{
    FirstName="Pjerrot",
    LastName="Pjerrotson",
    Age=500
};

string jsonObjectSerialized = JsonSerializer.Serialize(jsonObject);

Console.WriteLine(jsonObjectSerialized);

JsonObject j0De = JsonSerializer.Deserialize<JsonObject>(jsonObjectSerialized);

class JsonObject
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public int Age { get; set; }
}
```



Json i C# Eksempel Limitation!

- For at bruge den indbyggede serializer SKAL properties have Getter og setter!
 - Ellers forsvinder værdien!
 - Hvis man ikke vil have værdien serialiseret er det selvfølgelig en made at gøre det på 😬

```
using System.Text.Json;

JsonObject jsonObject= new JsonObject() {
    FirstName="Pjerrot", LastName="Pjerrotson", Age=500
};

string jsonObjectSerialized = JsonSerializer.Serialize(jsonObject);

Console.WriteLine(jsonObjectSerialized);

JsonObject j0De = JsonSerializer.Deserialize<JsonObject>(jsonObjectSerialized);
Console.WriteLine();

class JsonObject
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public int Age;
}
```

```
string jsonObjectSerialized = JsonSerializer.Serialize(jsonObject);
```

```
Console.WriteLine(jsonObjectSerialized);
```

```
JsonObject j0De = JsonSerializer.Deserialize<JsonObject>(jsonObjectSerialized);
Console.WriteLine();
```

jsonObject	{JsonObject}	
Age		500
FirstName	Q View	"Pjerrot"
LastName	Q View	"Pjerrotson"

```
string jsonObjectSerialized = JsonSerializer.Serialize(jsonObject);
```

```
Console.WriteLine(jsonObjectSerialized);
```

```
C:\Users\EmilS\source\repos\ X + v
{"FirstName": "Pjerrot", "LastName": "Pjerrotson"}
```

```
JsonObject j0De = JsonSerializer.Deserialize<JsonObject>(jsonObjectSerialized);
Console.WriteLine();

class JsonObject
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public int Age;
}
```

j0De	{JsonObject}	
Age		0
FirstName	Q View	"Pjerrot"
LastName	Q View	"Pjerrotson"

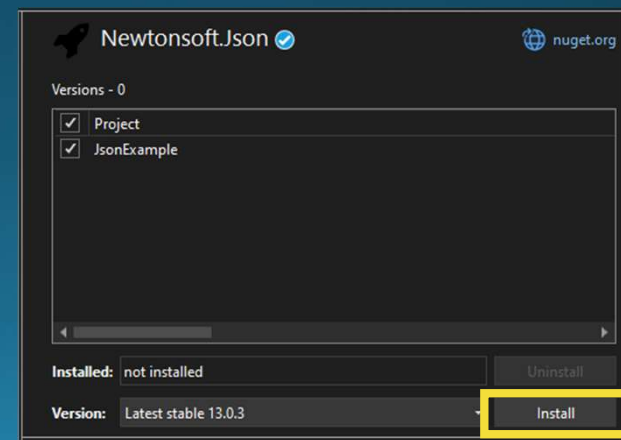
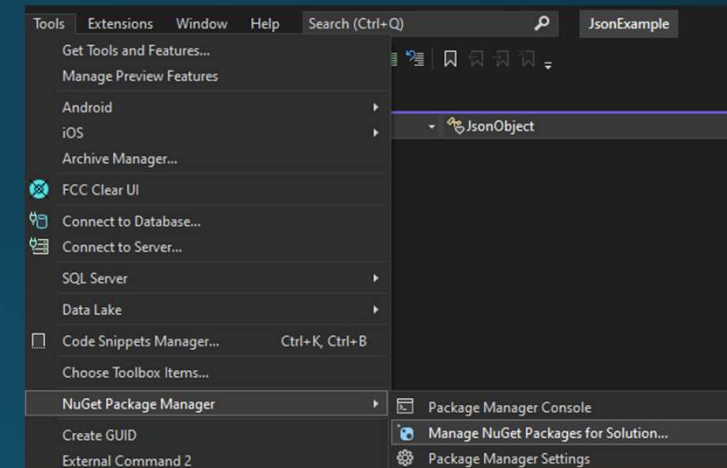
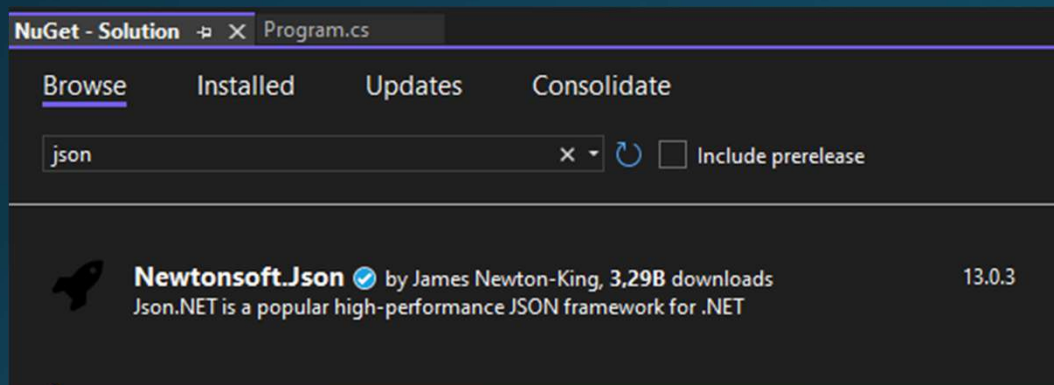
Bedre Json i C#

- Udover det indbyggede I c# findes der mange forskellige 3.parts biblioteker til håndtering af JSON formatet.
- Alle disse skal hentes som nuget pakker til det project de skal bruges til.
- Disse inkluderer:
 - Newtonsoft.Json (Json.NET)
 - Det mest anvendte Json bibliotek med 5.1B* downloads!
 - UTF8JSON
 - Et andet tredjepartsbibliotek, der fokuserer på ydeevne og lavt hukommelsesforbrug
 - Jil
 - Et andet hurtigt JSON-bibliotek med fokus på ydeevne og kompakt størrelse
- Største forskel på disse er API'en, hvor hurtige de udføre opgaven, og selvfølgelig hvor meget dokumentation de har! Alle ender i samme format.

*august 2024

JSON.net

- Hentes som nuget pakke til project
 - Tools-> nuGet Package Manager -> Manage Nuget...
 - Kan selvfølgelig også hentes via CLI, hvis man ønsker

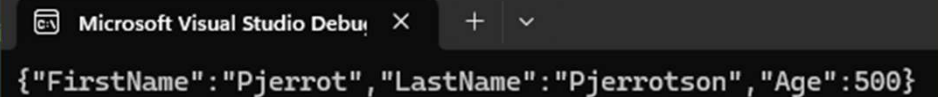


JSON.net Serialisering

- Serialize ligger nu i "JsonConvert.Serialize"
 - En del af Newtonsoft.Json namespace
- Behøver ikke være properties for at blive serialiseret!

```
class JsonObject
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public int Age;
}
```

```
using Newtonsoft.Json;
JsonObject jsonObject= new JsonObject()
{
    FirstName="Pjerrot",
    LastName="Pjerrotson",
    Age=500
};
string jsonnetOSerial = JsonConvert.SerializeObject(jsonObject);
Console.WriteLine(jsonnetOSerial);
```



Microsoft Visual Studio Debug Console

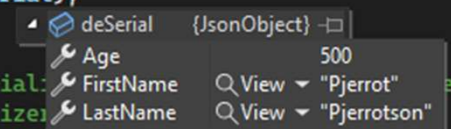
```
{"FirstName": "Pjerrot", "LastName": "Pjerrotson", "Age": 500}
```

JSON.net DeSerialisering

- Serialize ligger nu i "JsonConvert.Serialize"
- En del af Newtonsoft.Json namespace

```
class JsonObject
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public int Age { get; set; }
}
```

```
JsonObject deSerial = JsonConvert.DeserializeObject<JsonObject>(jsonnet0Serial);
Console.WriteLine(deSerial);
```



deSerial	{JsonObject}
Age	500
FirstName	View "Pjerrot"
LastName	View "Pjerrotson"

```
//string jsonObjectSerial: JsonConvert.SerializeObject(jsonObject);
//var j3s = JsonSerializer.Serialize(jsonObject);
//Console.WriteLine(j3s);
```

```
using Newtonsoft.Json;
JsonObject jsonObject= new JsonObject()
{
    FirstName="Pjerrot",
    LastName="Pjerrotson",
    Age=500
};

string jsonnet0Serial = JsonConvert.SerializeObject(jsonObject);
Console.WriteLine(jsonnet0Serial);

JsonObject deSerial = JsonConvert.DeserializeObject<JsonObject>(jsonnet0Serial);
```


JSON.net Attributes

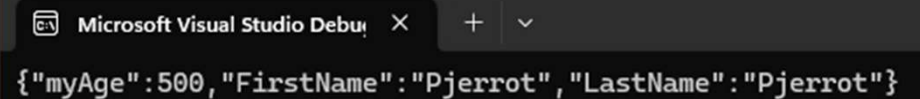
- Kan bruge attributes til at specificerer serialiseringen yderligere
- F.eks til private fields, og overskrivning af navne!
 - Obs, skal nu bruge samme navne property navne for at kunne deserialisere!

```
class JsonObject
{
    public string FirstName { get; set; }
    [JsonProperty]
    private string LastName=>FirstName;
    [JsonProperty("myAge")]
    public int Age;
}
```

```
using Newtonsoft.Json;

JsonObject jsonObject= new JsonObject()
{
    FirstName="Pjerrot",
    Age=500
};

string jsonnetOSerial = JsonConvert.SerializeObject(jsonObject);
Console.WriteLine(jsonnetOSerial);
```



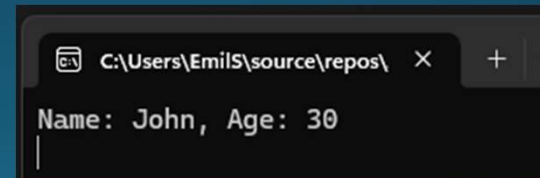
Microsoft Visual Studio Debug Console

```
{"myAge":500,"FirstName":"Pjerrot","LastName":"Pjerrot"}
```

JSON.net Anonyme typer

- Anonyme typer er dynamiske typer i C# uden en eksplicit typedefinition
- Nyttige til midlertidig deserialisering af JSON-data
 - Vi behøver ikke definere konkret klasse til deserialisering
- OBS: kaldes der noget på data der ikke findes i Json giver det exception.
 - Dvs prøver vi data.pjerrot fåes exception

```
string jsonData = "{ \"name\": \"John\", \"age\": 30 }";  
  
var data = JsonConvert.DeserializeObject<dynamic>(jsonData);  
  
string name = data.name;  
int age = data.age;  
  
Console.WriteLine($"Name: {name}, Age: {age}");
```



JSON.net Konfiguration

- Vi får her formatet til at være Indented eller “prettyprint”
- Ignorere null værdier (age)
- Konvertere vores enum som læsevenlig string
- Skifter vores casing fra pascal til camel.

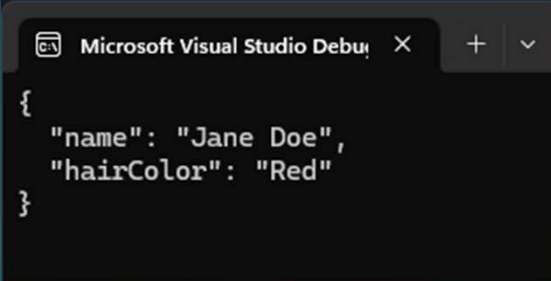
```
public class Person
{
    public string Name { get; set; }
    public int? Age { get; set; }
    public HairColor HairColor { get; set; }
}

public enum HairColor
{
    Brown,
    Black,
    Blonde,
    Red
}
```

```
JsonSerializerSettings settings = new JsonSerializerSettings
{
    Formatting = Formatting.Indented,
    NullValueHandling = NullValueHandling.Ignore,
    Converters = new List<JsonConverter> { new StringEnumConverter() },
    ContractResolver = new CamelCasePropertyNamesContractResolver()
};

var person = new Person
{
    Name = "Jane Doe",
    Age = null,
    HairColor = HairColor.Red
};

string json = JsonConvert.SerializeObject(person, settings);
Console.WriteLine(json);
```



```
{
  "name": "Jane Doe",
  "hairColor": "Red"
}
```

Skrivning og læsning af JSON-data med JSON.NET

- Vi kan bruge standard funktioner til at læse og skrive til en fil:

```
Person person = new Person { Name = "John Doe", Age = 30 };  
string json = JsonConvert.SerializeObject(person);  
File.WriteAllText("data.json", json);
```

- File oprettes blot i "bin" mappen
- Kan lige læses læse via:

```
string jsonR = File.ReadAllText("data.json");  
Person personR = JsonConvert.DeserializeObject<Person>(json);
```

```
public class Person  
{  
    public string Name { get; set; }  
    public int? Age { get; set; }  
    public HairColor HairColor { get; set; }  
}  
  
public enum HairColor  
{  
    Brown,  
    Black,  
    Blonde,  
    Red  
}
```

```
Person personR = JsonConvert.DeserializeObject<Person>(json);  
└─ personR {Person} ─┘  
Console.ReadLine() Age 30  
HairColor Brown  
Name John Doe  
//JsonSerializerSettings  
//{
```

Skrivning og læsning af JSON-data med JSON.NET – optimeret!

- Ved store datamængder kan det være mere effektivt at bruge JsonTextWriter og JsonTextReader direkte sammen med en StreamWriter eller StreamReader
 - Undgår også at allokere Json string i hukommelse

```
Person person = new Person { Name = "John Doe", Age = 30 };
using (StreamWriter streamWriter = new StreamWriter("data.json"))
using (JsonWriter jsonWriter = new JsonTextWriter(streamWriter))
{
    JsonSerializer serializer = new JsonSerializer();
    serializer.Serialize(jsonWriter, person);
}
Person personR;
using (StreamReader streamReader = new StreamReader("data.json"))
using (JsonReader jsonReader = new JsonTextReader(streamReader))
{
    JsonSerializer serializer = new JsonSerializer();
    personR = serializer.Deserialize<Person>(jsonReader);
}
```

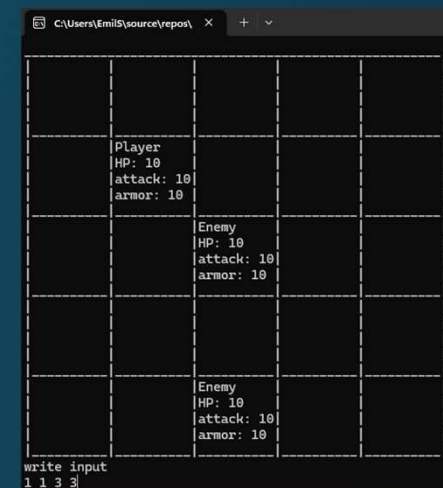
```
public class Person
{
    public string Name { get; set; }
    public int? Age { get; set; }
    public HairColor HairColor { get; set; }
}

public enum HairColor
{
    Brown,
    Black,
    Blonde,
    Red
}
```

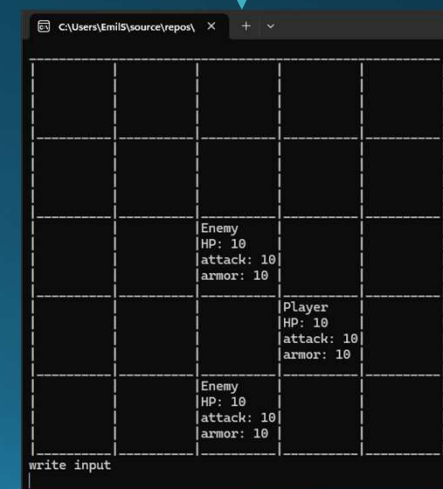
- Samme resultat som før, men bedre!

Øvelse 3 Json Som save format

- På moodle ligger et lille konsol spil
- Man kan bevæge de forskellige objekter via en kommando
- Udvid programmet så det har “save” og “load” funktioner ved at gemme og læse en json fil på disken.
- Tjek størrelsen på den gemte fil i bytes.
 - kan gøres i windows eller i c# programmet.
- Ekstra
 - Overvej hvordan det er muligt at gøre størrelsen på den gemte fil mindre, og implementer en løsning der bruger mindre plads.



1 1 3 3



JSON serialisering Minimering af størrelse

- Json har den smarte fordel at det er læseligt
- Hvad hvis vi ønsker at data skal fylde lidt ?
- Man kan godt med JSON, men kræver nogle forbedringer
- Der er noget spildt plads her...

```
{
  "type":0,
  "message":{
    "newPos":{
      "x":-1.0,
      "y":0.0,
      "z":0.0
    }
  }
}
```

```
{
  "type":1,
  "message":{
    "direction":{
      "x":0.0,
      "y":1.0,
      "z":0.0
    },
    "bulletType":1
  }
}
```

```
6 references
public enum MessageType { movement, shoot }
3 references
public enum BulletType { round, square }
[Serializable]
2 references
public class NetworkMessage
{
    public MessageType type;
    public NetworkMessageBase message;
}
[Serializable]
3 references
public class NetworkMessageBase
{
}
[Serializable]
6 references
public class PlayerMovementUpdate : NetworkMessageBase
{
    public Vector3 newPos;
}
[Serializable]
6 references
public class ShootUpdate : NetworkMessageBase
{
    public Vector3 direction;
    public BulletType bulletType;
}
```

JSON Minimering med skema

- Skal vi læse og skrive mange gange – f.eks på netværk ;)
 - Kan vi sende et stort skema én gang, der beskriver hvordan fremtidig data ser ud
- Herefter kan det sendes som json igen.
- ...Eller endnu mindre bare som en tekst streng, for at komme af med {}
 - Kræver endnu mere processering og kendskab af både læser og skriver.

```
6 references
public enum MessageType { movement, shoot }
3 references
public enum BulletType { round, square }
[Serializable]
2 references
public class NetworkMessage
{
    public MessageType type;
    public NetworkMessageBase message;
}
[Serializable]
3 references
public class NetworkMessageBase
{
}
[Serializable]
6 references
public class PlayerMovementUpdate : NetworkMessageBase
{
    public Vector3 newPos;
}
[Serializable]
6 references
public class ShootUpdate : NetworkMessageBase
{
    public Vector3 direction;
    public BulletType bulletType;
}
```

```
{
  "type": 0,
  "message": {
    "newPos": {
      "x": -1.0,
      "y": 0.0,
      "z": 0.0
    }
  }
}
```

```
{
  "type": 1,
  "message": {
    "direction": {
      "x": 0.0,
      "y": 1.0,
      "z": 0.0
    },
    "bulletType": 1
  }
}
```

```
{
  "movementUpdate": {
    "typeNum": "0",
    "properties": {
      "0": {
        "name": "newPos",
        "type": "vector3"
      }
    }
  },
  "shootUpdate": {
    "typeNum": "1",
    "properties": {
      "0": {
        "name": "direction",
        "type": "vector3",
        "1": {
          "name": "bulletType",
          "type": "int"
        }
      }
    }
  }
}
```

```
{
  "0": {
    "0": "(-1, 0, 0)"
  }
}
```

```
{
  "1": {
    "0": "(0, 1, 0)",
    "1": "1"
  }
}
```


Split af bytes

- Ved tekst baseret protokol bruger vi ofte kun A-Z,a-z, 0-9 og nogle få andre tegn
 - Dvs at vi kun bruger ~halvdelen af ascii table.
- Til store tal så som 21045, bruger vi 5 bytes, hvor i binær kunne vi have det i 16-bit(2bytes)(16bit int max størrelse af $2^{16}=65536$)
- Til små numre som 11, 31 eller 100 bruger vi en byte pr tal, hvor i binær kunne vi have det i 1 byte (op til 255)
- Vi spilder en hel byte til at repræsentere negative tal ved "-"
- Ligeledes bruger vi en hel byte på at lave "." i kommatall

ASCII TABLE

Decimal	Hexadecimal	Binary	Octal	Char	Decimal	Hexadecimal	Binary	Octal	Char	Decimal	Hexadecimal	Binary	Octal	Char
0	0	0	0	[NULL]	48	30	110000	60	0	96	60	1100000	140	.
1	1	1	1	[START OF HEADING]	49	31	110001	61	1	97	61	1100001	141	a
2	2	10	2	[START OF TEXT]	50	32	110010	62	2	98	62	1100010	142	b
3	3	11	3	[END OF TEXT]	51	33	110011	63	3	99	63	1100011	143	c
4	4	100	4	[END OF TRANSMISSION]	52	34	110100	64	4	100	64	1100100	144	d
5	5	101	5	[ENQUIRY]	53	35	110101	65	5	101	65	1100101	145	e
6	6	110	6	[ACKNOWLEDGE]	54	36	110110	66	6	102	66	1100110	146	f
7	7	111	7	[BELL]	55	37	110111	67	7	103	67	1100111	147	g
8	8	1000	10	[BACKSPACE]	56	38	111000	70	8	104	68	1101000	150	h
9	9	1001	11	[HORIZONTAL TAB]	57	39	111001	71	9	105	69	1101001	151	i
10	A	1010	12	[LINE FEED]	58	3A	111010	72	:	106	6A	1101010	152	j
11	B	1011	13	[VERTICAL TAB]	59	3B	111011	73	;	107	6B	1101011	153	k
12	C	1100	14	[FORM FEED]	60	3C	111100	74	<	108	6C	1101100	154	l
13	D	1101	15	[CARRIAGE RETURN]	61	3D	111101	75	=	109	6D	1101101	155	m
14	E	1110	16	[SHIFT OUT]	62	3E	111110	76	>	110	6E	1101110	156	n
15	F	1111	17	[SHIFT IN]	63	3F	111111	77	?	111	6F	1101111	157	o
16	10	10000	20	[DATA LINK ESCAPE]	64	40	1000000	100	@	112	70	1100000	160	p
17	11	10001	21	[DEVICE CONTROL 1]	65	41	1000001	101	A	113	71	1100001	161	q
18	12	10010	22	[DEVICE CONTROL 2]	66	42	1000010	102	B	114	72	1100010	162	r
19	13	10011	23	[DEVICE CONTROL 3]	67	43	1000011	103	C	115	73	1100011	163	s
20	14	10100	24	[DEVICE CONTROL 4]	68	44	1000100	104	D	116	74	1100100	164	t
21	15	10101	25	[NEGATIVE ACKNOWLEDGE]	69	45	1000101	105	E	117	75	1100101	165	u
22	16	10110	26	[SYNCHRONOUS IDLE]	70	46	1000110	106	F	118	76	1100110	166	v
23	17	10111	27	[ENG OF TRANS. BLOCK]	71	47	1000111	107	G	119	77	1100111	167	w
24	18	11000	30	[CANCEL]	72	48	1001000	110	H	120	78	1110000	170	x
25	19	11001	31	[END OF MEDIUM]	73	49	1001001	111	I	121	79	1110001	171	y
26	1A	11010	32	[SUBSTITUTE]	74	4A	1001010	112	J	122	7A	1110010	172	z
27	1B	11011	33	[ESCAPE]	75	4B	1001011	113	K	123	7B	1110011	173	{
28	1C	11100	34	[FILE SEPARATOR]	76	4C	1001100	114	L	124	7C	1111000	174	
29	1D	11101	35	[GROUP SEPARATOR]	77	4D	1001101	115	M	125	7D	1111001	175	}
30	1E	11110	36	[RECORD SEPARATOR]	78	4E	1001110	116	N	126	7E	1111010	176	~
31	1F	11111	37	[UNIT SEPARATOR]	79	4F	1001111	117	O	127	7F	1111110	177	[DEL]
32	20	100000	40	[SPACE]	80	50	1010000	120	P					
33	21	100001	41	!	81	51	1010001	121	Q					
34	22	100010	42	"	82	52	1010010	122	R					
35	23	100011	43	#	83	53	1010011	123	S					
36	24	100100	44	\$	84	54	1010100	124	T					
37	25	100101	45	%	85	55	1010101	125	U					
38	26	100110	46	&	86	56	1010110	126	V					
39	27	100111	47	'	87	57	1010111	127	W					
40	28	101000	50	(	88	58	1011000	130	X					
41	29	101001	51	)	89	59	1011001	131	Y					
42	2A	101010	52	*	90	5A	1011010	132	Z					
43	2B	101011	53	+	91	5B	1011011	133	[					
44	2C	101100	54	,	92	5C	1011100	134	\					
45	2D	101101	55	-	93	5D	1011101	135	]					
46	2E	101110	56	.	94	5E	1011110	136	^					
47	2F	101111	57	/	95	5F	1011111	137	_					

Drop Det – Gå biner

- Man kan gøre meget når man er nede i den tætte opkopling og målet er minimering af dataets størrelse.
- Tekst er ineffektivt, der findes andre serieliserings protokoller end Json og lign, så som BSON, **messagepack**, protobuf og flatbuffer, men intet slår en custom binær protokol.
- Det kræver en god håndfuld viden omkring bit manipulation, hvis i gerne vil vide mere så tjek:
https://gafferongames.com/post/reading_and_writing_packets/

custom binær serielisering- Eksempel

- Kigger vi på en simpel struct kan vi se hvad den *kan* fylde
- En byte + en bool (1 byte) + float (4 bytes) bør være 6 bytes
- Så er det egentligt blot et spørgsmål om at gemme dette som et byte array og vide hvordan det skal læses.
- Vi kan benytte marshalling til at opnå dette.
 - Et API i .net
 - Marshalling er egentlig lavet til at konvertere mellem managed code (c# og lign) til unmanaged code (assembly, c++,c etc.)

```
public struct MyDataStruct
{
    public byte Id;
    public float Value;
    public bool Enabled;
}
```

custom binær serielisering- Eksempel

- Vi starter med at fortælle at vores struct skal gemmes som en sekventiel struktur
- Pack 1 betyder at vi ikke ønsker nogen form for padding mellem de forskellige bytes.
- Dvs. det kommer til at være et tæt pakket array af bytes.

```
[StructLayout(LayoutKind.Sequential, Pack = 1)]
public struct MyDataStruct
{
    public byte Id;
    public float Value;
    [MarshalAs(UnmanagedType.I1)]
    public bool Enabled;

    public static byte[] Serialize(MyDataStruct data)
    {
        int size = Marshal.SizeOf(data);
        byte[] buffer = new byte[size];
        IntPtr ptr = Marshal.AllocHGlobal(size);
        Marshal.StructureToPtr(data, ptr, true);
        Marshal.Copy(ptr, buffer, 0, size);
        Marshal.FreeHGlobal(ptr);
        return buffer;
    }
}
```

custom binær serielisering- Eksempel

- MarshalAs attributten fortæller den marshaller at det skal marshalles med specifikke regler.
- Pr. default konverterer C# bool værdier til 4 bytes
 - Vi kan konfigurere til at bruge typen "I1"

I1 3 A 1-byte signed integer. You can use this member to transform a Boolean value into a 1-byte, C-style `bool` (`true` = 1, `false` = 0).

<https://learn.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.unmanagedtype?view=net-8.0>

```
[StructLayout(LayoutKind.Sequential, Pack = 1)]
public struct MyDataStruct
{
    public byte Id;
    public float Value;
    [MarshalAs(UnmanagedType.I1)]
    public bool Enabled;

    public static byte[] Serialize(MyDataStruct data)
    {
        int size = Marshal.SizeOf(data);
        byte[] buffer = new byte[size];
        IntPtr ptr = Marshal.AllocHGlobal(size);
        Marshal.StructureToPtr(data, ptr, true);
        Marshal.Copy(ptr, buffer, 0, size);
        Marshal.FreeHGlobal(ptr);
        return buffer;
    }
}
```

custom binær serielisering- Eksempel

- Selve serialiserings koden
 - Note: lavet som static, så man ikke behøver bruge en instans af struct'en for at bruge
- Vi finder størrelsen ved `Marshal.SizeOf(data)`
 - Bør i dette tilfælde være 6 bytes 😊
- Opretter nyt byte array på denne længde
- Allokere plads i hukommelsen til pointer
 - Dvs. en reference til hvor i vores hukommelse der er plads til at gemme præcis dette.

```
[StructLayout(LayoutKind.Sequential, Pack = 1)]
public struct MyDataStruct
{
    public byte Id;
    public float Value;
    [MarshalAs(UnmanagedType.I1)]
    public bool Enabled;

    public static byte[] Serialize(MyDataStruct data)
    {
        int size = Marshal.SizeOf(data);
        byte[] buffer = new byte[size];
        IntPtr ptr = Marshal.AllocHGlobal(size);
        Marshal.StructureToPtr(data, ptr, true);
        Marshal.Copy(ptr, buffer, 0, size);
        Marshal.FreeHGlobal(ptr);
        return buffer;
    }
}
```

custom binær serielisering- Eksempel

- Kopierer struct'en som byte array til hukommelsen
 - True sletter evt data i Ptr før det nye kopieres
- Kopierer vores data fra unmanaged data i hukommelsen over i vores byte array
- Frigiver hukommelsen
- Returnerer array'et.

```
[StructLayout(LayoutKind.Sequential, Pack = 1)]
public struct MyDataStruct
{
    public byte Id;
    public float Value;
    [MarshalAs(UnmanagedType.I1)]
    public bool Enabled;

    public static byte[] Serialize(MyDataStruct data)
    {
        int size = Marshal.SizeOf(data);
        byte[] buffer = new byte[size];
        IntPtr ptr = Marshal.AllocHGlobal(size);
        Marshal.StructureToPtr(data, ptr, true);
        Marshal.Copy(ptr, buffer, 0, size);
        Marshal.FreeHGlobal(ptr);
        return buffer;
    }
}
```

Deserialisering af struct.

- Deserialiseringen udfører stort set det samme, men i omvendt rækkefølge. 😊

```
[StructLayout(LayoutKind.Sequential, Pack = 1)]
public struct MyDataStruct
{
    public byte Id;
    public float Value;
    [MarshalAs(UnmanagedType.I1)]
    public bool Enabled;

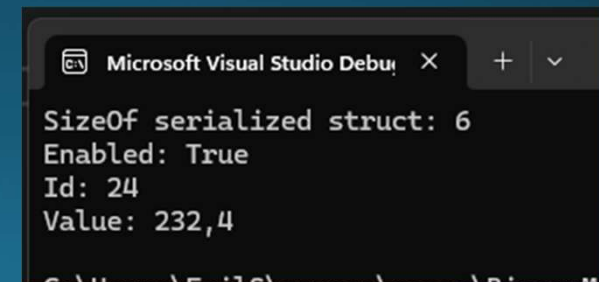
    public static byte[] Serialize(MyDataStruct data)
    {
        int size = Marshal.SizeOf(data);
        byte[] buffer = new byte[size];
        IntPtr ptr = Marshal.AllocHGlobal(size);
        Marshal.StructureToPtr(data, ptr, true);
        Marshal.Copy(ptr, buffer, 0, size);
        Marshal.FreeHGlobal(ptr);
        return buffer;
    }

    public static MyDataStruct Deserialize(byte[] buffer)
    {
        MyDataStruct data = new MyDataStruct();
        int size = Marshal.SizeOf(data);
        IntPtr ptr = Marshal.AllocHGlobal(size);
        Marshal.Copy(buffer, 0, ptr, size);
        data = (MyDataStruct)Marshal.PtrToStructure(ptr,
        typeof(MyDataStruct));
        Marshal.FreeHGlobal(ptr);
        return data;
    }
}
```


Begge vejs

- Programmet opfører sig som forventet.

```
MyDataStruct dataStruct= new MyDataStruct() {  
    Enabled=true, Id=24, Value=232.4f};  
byte[] structAsBytes = MyDataStruct.Serialize(dataStruct);  
Console.WriteLine("SizeOf serialized struct: " +structAsBytes.Length);  
  
MyDataStruct deserialStruct = MyDataStruct.Deserialize(structAsBytes);  
Console.WriteLine("Enabled: "+ dataStruct.Enabled);  
Console.WriteLine("Id: " + dataStruct.Id);  
Console.WriteLine("Value: " + dataStruct.Value);
```



Microsoft Visual Studio Debug Console output:

```
SizeOf serialized struct: 6  
Enabled: True  
Id: 24  
Value: 232,4
```

Ulemper ved Custom Binær Serialisering

- Høj risiko for fejl:
 - Kræver dyb forståelse af datatyper, hukommelse, endianess osv.
- Ikke en generisk løsning:
 - Nødvendigt at skrive separate serializations for HVER type objekt.
- Komplekse datatyper:
 - Svært at håndtere komplekse objekter, dyb nesting og referencehåndtering.

binær serielisering - formater

- I stedet for at skrive fra bunden kan vi bruge eksisterende formater dette giver os en håndfuld af fordele
- Effektivitet og ydeevne:
 - Optimerede til at producere kompakte binære repræsentationer og sikre hurtig serialisering og deserialisering.
- Platformuafhængighed:
 - Kan bruges på tværs af forskellige programmeringssprog og platforme.
- Skemaevolution og bagudkompatibilitet:
 - Støtter ændringer i datastrukturer uden at bryde eksisterende datakompatibilitet.
- Tilgængelige værktøjer og biblioteker:
 - Gør det nemt at generere kode, håndtere serialisering og deserialisering samt integrere med eksisterende systemer og frameworks.
- Fællesskab og support:
 - Stort udviklerfællesskab med dokumentation, eksempler, forummer og support til rådighed.

binær serialisering - formater

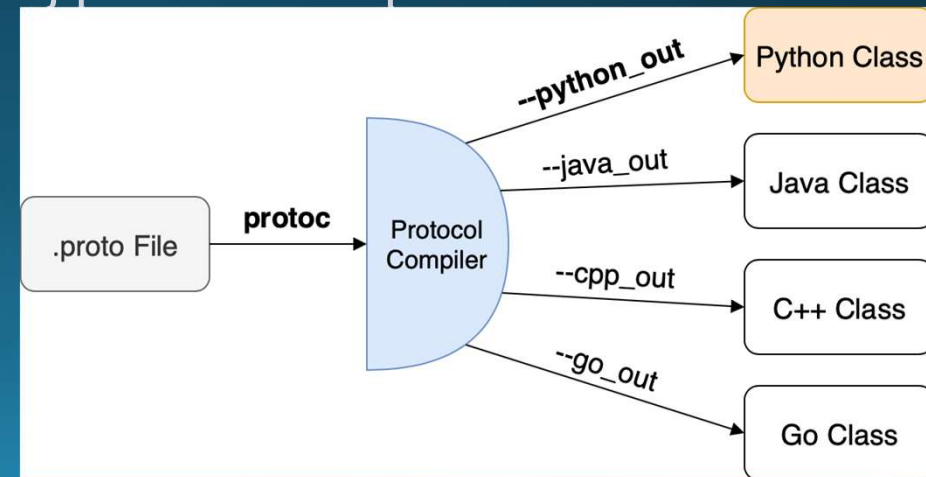
- Der findes mange forskellige binære serialiseringsformater tilgængelige, der kan hjælpe med at effektivisere dataudveksling mellem systemer og applikationer.
- Når man vælger et binært serialiseringsformat, er det vigtigt at overveje følgende faktorer:
 1. Datastørrelse:
 - Er det vigtigt, at dataen fylder så lidt som muligt? Mindre størrelse betyder mindre båndbredde og diskplads.
 2. Ydeevne:
 - Er det vigtigt, at dataen er hurtig at deserialisere eller serialisere? Ydeevne kan være afgørende, især ved højvolumen og realtidsapplikationer.
 3. Skemaevolution:
 - Har du brug for et serialiseringsformat, der kan håndtere skemaændringer over tid uden at bryde eksisterende data? Skemaevolution er vigtigt, hvis du forventer, at dine datastrukturer vil ændre sig over tid.
 4. Platformunderstøttelse:
 - Har formatet implementeringer og understøttelse på tværs af forskellige programmeringssprog og platforme? Det er vigtigt at vælge et format, der er kompatibelt med dine eksisterende teknologistak og udviklingsmiljø.
- Afhængig af use case og krav kan det være svært at finde et format, der opfylder alle ens behov. Derfor er det vigtigt at prioritere de mest afgørende faktorer for ens applikation.

binær serielisering - formater

- Lad os kigge på 3 forskellige formater:
 - Protocol Buffers
 - Flatbuffers
 - MessagePack

Protocol Buffers (protobuf)

- Udviklet af Google
- Definerer skemaer ved hjælp af .proto-filer skema
- Effektiv binær serialisering med lille størrelse og hurtig parsing
- Understøtter versionering af skemaer med bagudkompatibilitet
- C#-implementering: Google.Protobuf og protoc compileren
- Genererer C#-klasser fra .proto-filer
- <https://protobuf.dev/>



FlatBuffers

- Udviklet af Google
- Høj ydeevne og effektiv dataadgang
- Tillader direkte adgang til serialiserede data uden de-serialisering
- Populært inden for systemer med krav til ydeevne og lav latens
- C#-implementering:
 - FlatBuffers.NET
- Genererer C#-klasser fra FlatBuffers-skemaer
- Nyere end protobuf – i mange tilfælde bedre, men også sværere at arbejde med og har mindre dokumentation.
- <https://flatbuffers.dev/>



Fig. 4. Diagram flatbuffers

Mest bruger venlige --> brug den

MessagePack

- Kompakt binær serialisering med fokus på ydeevne
- Understøtter automatisk serialisering og deserialisering af C#-objekter
- Kan integreres i eksisterende C#-applikationer uden at skulle definere skemaer
- Støtter mange typer af C#-objekter, inklusive komplekse datastrukturer
- C#-implementering:
 - MessagePack-Csharp
- Intet skema, så format skal være kendt af alle programmer, især relevant hvis man bruger forskellige sprog!
- <https://github.com/neuecc/MessagePack-CSharp>

Valg af serialiseringsformat

- Json:
 - Når læsbarhed og nem menneskelig forståelse af data er vigtig.
 - Ved dataudveksling mellem systemer, der ikke har en fælles skemadefinition
 - Dvs man potentielt ikke ved hvem der vil bruge ens applikation.
 - Når det er nødvendigt at arbejde med data i forskellige programmeringssprog
 - Det mest populære, og det mest brugte på internettet.
- MessagePack:
 - Når pladsbesparelse og ydeevne er afgørende faktorer
 - Ved kommunikation mellem systemer med høj dataoverførselsmængde og lav båndbredde – eks kommunikation med gameserver 😊
 - Når der er behov for hurtig serialisering og deserialisering af data
 - Nyt, hurtigt, godt til tæt koblede programmer der kender hinanden.

Valg af binært serialiseringsformat

- Protobuf:
 - Når effektivitet, skemastyret kommunikation og versionering er vigtige
 - Ved store og komplekse datamodeller, der kræver skemadefinition og skemaudvidelse
 - Ved arbejde med distribuerede systemer og mikrotjenestearkitekturer
 - Mere support/dokumentation end flatbuffers
- FlatBuffers:
 - Også skema baseret
 - Når lav latency og direkte adgang til data er afgørende
 - Ved store datamængder, der skal håndteres effektivt uden kopiering
 - Store datamængder som i at læse fra disken uden at bruge ram

Øvelse 4 - binær serialisering

- Vælg en af de 3 binære serialiseringsformater
 - Messagepack er den nemmeste at arbejde med.
- Implementer binær serialisering, i stedet for Json
 - Undersøg forskellen i det gemte data.
 - Jeg fik det ned på 25bytes(med messagepack) med 1 player og 2 enemies + variabel størrelse på griddet. Kan denne slås?